Laboratoire de validation AAM O94_3

Dossier technique AAM

Version 0.94.3 — 19 octobre 2026

Contents

1 Description et objectif **1**

2 Architecture du laboratoire **1**

3 Pipeline complet **2**

4 Commandes d'exécution **2**

5 Résultats et interprétation initiale **2**
- 5.1 Préservation du RAG . . . 2
- 5.2 Observation passive . . . 3
- 5.3 Apprentissage des briques . . . 3

6 Contrat ergonomique **3**

7 Résultats finaux constatés et reçus de preuve **3**

8 Conséquences théoriques, conceptuelles et opérationnelles **4**

9 Conclusion et statut **4**

1 Description et objectif

Ce laboratoire vérifie les anomalies rapportées sur O94_2 : le bouton Observer s'appuyait sur un chemin de JSONL pré-existant; la capture passive n'était pas distincte du sondage actif; les onglets denses étaient difficiles à parcourir; les adaptateurs de modèles locaux étaient peu visibles; la compilation et l'apprentissage de briques n'étaient pas assez exposés; et il fallait prouver la préservation du RAG historique.

Le test ne cherche pas à déclarer l'objectif global AAM réussi ou échoué à l'avance. Chaque cas a un périmètre, un budget, une commande, un résultat et une limite clairement déclarés.

2 Architecture du laboratoire

Les essais sont séparés en cinq familles: (i) comparaison des empreintes des archives O94_0/O94_1/O94_2, (ii) capture filesystem/process/réseau, (iii) lecture des traces et apprentissage, (iv) interface/scroll, modèles et composition, (v) régression RAG et réception du paquet. Le répertoire `receipts/O94_3` garde les résultats des commandes. Les incidents non résolus restent marqués comme tels; ils ne sont jamais effacés par une nouvelle tentative.

3 Pipeline complet

1. Relever les fichiers source, hashes des archives O94_0 à O94_2 et hash du moteur RAG, des lecteurs O92 et de leurs tests.
2. Créer O94_3 comme une incrémentation de O94_2; conserver les fichiers RAG historiques sans modification.
3. Démarrer une observation passive depuis l'UI ou le CLI, sans fournir de JSONL préalable. Observer le dossier ou le PID sélectionné, ou les paquets visibles via TShark.
4. Contrôler que le JSONL a été créé par AAM, que les séquences et hashes se valident, que le rapport final existe et que l'arrêt respecte le budget.
5. Apprendre les briques depuis une trace: exact-lookup seulement pour les entrées/sorties train non ambiguës; méthodes/règles passives restent candidates à examiner.
6. Compiler une composition DSL via le catalogue des compétences, obtenir un artefact versionné, puis tester son comportement séparément.
7. Tester les espaces de modèles Transformers/PyTorch/Ollama/OpenAI-compatible et leur comportement lorsque leur serveur ou dépendance n'est pas disponible.
8. Rejouer les tests d'interface, de modèle, de composition, de RAG et de l'ancien corpus; consigner les commandes interrompues comme non concluantes.
9. Rafraîchir le savepoint et les manifests, produire les archives, les extraire dans un dossier neuf et refaire les contrôles d'intégrité.

4 Commandes d'exécution

```
python -m compileall -q aam_o93 tests
python -m pytest -q tests/test_o943_passive_observation.py
python -m pytest -q tests/test_o943_learned_blocks_and_composition.py
python -m pytest -q tests/test_p3_semantic_rag.py tests/test_o940_rag_resolution.py
python -m pytest -q tests/test_o940_real_sms_retrieval.py tests/test_o94_rag_adaptive.py
python -m pytest -q tests/test_o943_ui_contracts.py
python -m aam_o93 preflight
python -m aam_o93 observe-live --source-kind filesystem \\
  --root ./tmp_target --authorize-source --seconds 5 --interval .05
```

Le test réseau peut déclarer `DEPENDENCY_MISSING` quand TShark n'est pas installé; cette situation doit fournir un diagnostic précis et n'est pas maquillée en succès de capture réseau.

5 Résultats et interprétation initiale

5.1 Préservation du RAG

Le comparatif d'archives confirme des hashes SHA-256 identiques pour `aam_o93/semantic_rag.py`, `aam_o92/retrieval.py`, `test_p3_semantic_rag.py`, `test_o940_rag_resolution.py`, `test_o940_real_sms_r` et `test_o94_rag_adaptive.py` à travers les trois archives parent O94_0, O94_1 et O94_2. Le rapport machine vérifiable est `rapport_O94_3/rag_parity_O94_0_O94_1_O94_2.json`. Les fichiers ne sont pas modifiés par O94_3.

5.2 Observation passive

Le test de base crée lui-même un JSONL dans un répertoire de sortie, modifie le fichier observé pendant la session, puis vérifie les événements ajout/modification/suppression, les numéros de séquence strictement croissants, la chaîne de hashes, le rapport d'arrêt et la fermeture du worker. Les sorties chiffrées par reçus distinguent le nombre d'événements observés des lignes techniques non comptabilisées dans ce budget.

5.3 Apprentissage des briques

Le test train/holdout vérifie que les cas de test/validation/holdout ne deviennent pas des correspondances exécutables. Les entrées ambiguës sont exclues; une entrée non vue doit retourner `UNRESOLVED`. Une trace passive génère un profil de règle ou de méthode marqué `MANUAL_REVIEW_REQUIRED` et ne prétend pas avoir appris un clone exécutable à partir de simples événements.

6 Contrat ergonomique

Les onglets denses sont logés dans un canvas avec une barre verticale. Les éditeurs JSON/textes disposent de barres distinctes. Pour l'observation live, le formulaire expose la source, PID/interface, budgets, filtre réseau, dossier de sortie automatique et opt-in payload. Pour modèles, le panneau expose Transformers local, PyTorch TorchScript tensoriel, adaptateur Python PyTorch pour architecture connue, Ollama et OpenAI-compatible, ainsi que le workspace AAM et les politiques associées. Le gabarit PyTorch est toujours disponible pour les `state_dict` dépendants d'une architecture. Le compilateur de composition écrit un artefact JSON dans le workspace et la liste des briques reflète les candidats observés.

7 Résultats finaux constatés et reçus de preuve

Test	Statut documenté	Interprétation
Empreintes RAG parent	`PASS_BOUNDED`	Six fichiers protégés correspondent par SHA-256 à O94_2; les archives parentes O94_0/O94_1/O94_2 ont été comparées. Voir P0/P5.
Capture filesystem créée par AAM	`PASS_BOUNDED`	Le CLI crée son JSONL et son rapport sans fichier de trace préalable; le test ajoute/modifie/supprime un fichier pendant la session. Voir P1.
Capture process PID	`PASS_BOUNDED`	Instantané du processus courant testé; l'accès dépend de psutil et des permissions. Voir P1.
Capture réseau TShark	`FIXTURE_PASS`	Chemin PCAPNG/parser testé avec fixtures; aucune capture réelle sur une interface réseau n'est revendiquée dans cet environnement. Voir P1.
Apprentissage / train-holdout	`PASS_BOUNDED`	Correspondance exacte sur support train non ambigu; cas tenus à l'écart exclus; entrée jamais vue `UNRESOLVED`. Voir P4.
UI scroll	`PASS_BOUNDED`	Conteneurs avec barre verticale et molette testés sous affichage virtuel; rendu natif Windows non exécuté ici. Voir P2.

Modèles locaux	`PASS_BOUNDED`	Module TorchScript local synthétique chargé et exécuté sur CPU; contrats Ollama/API compatibles testés par fixtures, sans endpoint externe réel. Voir P3.
Compilation de composition	`PASS_BOUNDED`	Le DSL compile un artefact JSON versionné, rejette les références inconnues et n'exécute pas du code arbitraire. Voir P4.
RAG / SMS	`PASS_BOUNDED`	47 tests passés; six fichiers historiques restent identiques par SHA-256. Voir P5.
Régression intégrée	`PASS_BOUNDED`	15 lots passés couvrent exactement 60 fichiers de tests uniques, sans doublon ni omission; 321 tests et 21 sous-tests réussis. Voir P6.
Compilation Python	`PASS_BOUNDED`	`compileall` a analysé 328 fichiers Python. Voir P6.
Packaging après extraction fraîche	Reçu P8 + manifeste externe	CRC, chemins, inventaire, hashes du contenu et tests essentiels sont consignés dans le rapport externe de livraison afin d'éviter une empreinte circulaire dans le ZIP du modèle.

8 Conséquences théoriques, conceptuelles et opérationnelles

Théoriques : O94_3 sépare observation d'événements OS, test par interface active et jeu de données prêt à l'emploi. Les limites d'instrumentation deviennent des propriétés explicites et non un résultat fabriqué.

Conceptuelles : une trace d'événements ne suffit pas à déduire l'implémentation entière d'un objet; elle produit des invariants observés et des candidats. Les briques exécutables à support fini doivent rester distinctes des méthodes/règles proposées.

Opérationnelles : AAM crée désormais ses traces d'observation filesystem/PID lui-même, fournit des budgets et produit un rapport d'arrêt; les traces réseau dépendent d'un outil externe. Les surfaces de modèles, de compilation et de défilement doivent être vérifiées depuis l'archive, pas seulement le répertoire source.

9 Conclusion et statut

Le statut final de chaque phase est dans `checklist_de_travail_O94_3.json`; les échecs et retests sont dans les registres d'incidents et reçus correspondants. La livraison n'affirme pas l'observation invisible et totale de n'importe quelle application, l'accès arbitraire aux mémoires de processus ou le clonage universel d'un logiciel. Elle fournit des adaptateurs observables, un protocole d'intégration explicite et des critères reproductibles pour étendre les niveaux de capture.